

Proxmox VE + Browser Isolation - Complete Setup Guide

Goal: Run isolated browser in Proxmox VM with GUI via SPICE/noVNC

Requirements:

- Dedicated machine OR VM with nested virtualization
- 16GB RAM minimum (Proxmox itself needs 4GB)
- 100GB disk space
- Intel VT-x/AMD-V support

Note: Proxmox is a full hypervisor platform - different approach than others

Part 1: Install Proxmox VE

Step 1.1: Download Proxmox

```
bash

# Download from: https://www.proxmox.com/en/downloads

# Proxmox VE 8.1 ISO (latest as of 2024)
wget https://enterprise.proxmox.com/iso/proxmox-ve_8.1-1.iso

# Burn to USB:
# On Linux:
sudo dd if=proxmox-ve_8.1-1.iso of=/dev/sdX bs=1M status=progress
# Replace /dev/sdX with your USB device!

# On Windows: Use Rufus or Etcher
```

Step 1.2: Install Proxmox

1. Boot from USB
2. Select "Install Proxmox VE (Graphical)"
3. Accept EULA
4. Select target disk
5. Set timezone and keyboard layout
6. Set password and email
7. Configure network:
 - Hostname: pve.local
 - IP: 192.168.1.100/24 (adjust to your network)
 - Gateway: 192.168.1.1 (your router)
 - DNS: 8.8.8.8
8. Install (10-15 minutes)
9. Reboot

Step 1.3: Access Proxmox Web UI

```
bash

# From another computer on same network
# Open browser and go to:
https://192.168.1.100:8006

# Login:
# Username: root
# Password: (what you set during install)

# Accept self-signed certificate warning
```

Part 2: Initial Proxmox Configuration

Step 2.1: Remove Enterprise Repository (Community Use)

```
bash
```

```
# SSH into Proxmox server
```

```
ssh root@192.168.1.100
```

```
# Comment out enterprise repo
```

```
sed -i 's/^deb/#deb/' /etc/apt/sources.list.d/pve-enterprise.list
```

```
# Add community repo
```

```
echo "deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription" > /et
```

```
# Update
```

```
apt update && apt upgrade -y
```

```
# Reboot
```

```
reboot
```

Step 2.2: Upload ISO Images (Via Web UI)

1. In Proxmox web UI: Click "pve" (server name)
2. Click "local (pve)"
3. Click "ISO Images"
4. Click "Upload"
5. Upload Ubuntu 22.04 Desktop ISO
Download from: <https://ubuntu.com/download/desktop>

Part 3: Create Browser Isolation VM

Step 3.1: Create VM via Web UI

1. Click "Create VM" (top right)
2. General Tab:
 - VM ID: 100
 - Name: browser-isolation
 - Click Next
3. OS Tab:
 - ISO image: ubuntu-22.04-desktop-amd64.iso
 - Type: Linux
 - Version: 6.x - 2.6 Kernel
 - Click Next
4. System Tab:
 - Graphics card: SPICE
 - Machine: q35
 - BIOS: OVMF (UEFI)
 - Add EFI Disk: Yes
 - SCSI Controller: VirtIO SCSI
 - Click Next
5. Disks Tab:
 - Bus/Device: VirtIO Block (scsi0)
 - Storage: local-lvm
 - Disk size: 32 GB
 - Cache: Write back
 - Click Next
6. CPU Tab:
 - Cores: 2
 - Type: host
 - Click Next
7. Memory Tab:
 - Memory: 4096 MB (4GB)
 - Click Next
8. Network Tab:
 - Bridge: vmbr0
 - Model: VirtIO
 - Click Next
9. Confirm Tab:
 - Review settings
 - Check "Start after created"
 - Click Finish

Step 3.2: Install Ubuntu Desktop

1. VM will auto-start
 2. Click "Console" to see installation
 3. Follow Ubuntu installer:
 - Try or Install Ubuntu → Install Ubuntu
 - Keyboard layout → Your layout
 - Updates and software → Normal installation
 - Installation type → Erase disk and install
 - Create user:
 - Name: browser
 - Computer: browser-isolation
 - Username: browser
 - Password: browser123
 4. Wait for installation (15-20 minutes)
 5. Reboot when prompted
 6. Login
-

Part 4: Configure VM for Browser Isolation

Step 4.1: Install Required Software (Inside VM)

```
bash
```

```
# Open terminal in Ubuntu desktop
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Install browsers and tools
```

```
sudo apt install -y \  
    chromium-browser \  
    firefox \  
    qemu-guest-agent \  
    spice-vdagent \  
    openssh-server \  
    inotify-tools \  
    python3-watchdog
```

```
# Enable services
```

```
sudo systemctl enable qemu-guest-agent
```

```
sudo systemctl start qemu-guest-agent
```

```
sudo systemctl enable ssh
```

```
# Configure auto-login (optional)
```

```
sudo mkdir -p /etc/gdm3/
```

```
sudo tee -a /etc/gdm3/custom.conf << EOF
```

```
[daemon]  
AutomaticLoginEnable=true  
AutomaticLogin=browser  
EOF
```

```
# Reboot to apply
```

```
sudo reboot
```

Step 4.2: Optimize for Isolation

```
bash
```

```
# Disable unnecessary services
```

```
sudo systemctl disable bluetooth
```

```
sudo systemctl disable cups
```

```
sudo systemctl disable avahi-daemon
```

```
# Configure firewall
```

```
sudo ufw enable
```

```
sudo ufw allow ssh
```

```
sudo ufw allow from 192.168.1.0/24
```

```
# Create downloads monitoring folder
```

```
mkdir -p ~/Downloads
```

```
mkdir -p ~/Downloads-Safe
```

Part 5: Access Methods

Step 5.1: SPICE Console (Best Quality)

```
Method 1 - Web Console (noVNC):
```

1. In Proxmox UI → Select VM "browser-isolation"
2. Click "Console" button
3. Browser displays VM screen directly
4. Click inside to capture keyboard/mouse

```
Method 2 - SPICE Client (Better Performance):
```

```
On your host computer:
```

```
# Linux
```

```
sudo apt install virt-viewer
```

```
remote-viewer spice://192.168.1.100:61000
```

```
# Windows
```

```
Download virt-viewer from:
```

```
https://virt-manager.org/download/
```

```
Run: remote-viewer.exe spice://192.168.1.100:61000
```

```
# macOS
```

```
brew install virt-viewer
```

```
remote-viewer spice://192.168.1.100:61000
```

Step 5.2: VNC Access (Alternative)

```
bash
```

```
# If SPICE doesn't work, enable VNC:
```

```
# In Proxmox UI:
```

1. Select VM → Hardware
2. Click "Display"
3. Change to "VNC"
4. Click OK
5. Restart VM

```
# Connect with any VNC client:
```

```
vncviewer 192.168.1.100:5900
```

Part 6: File Monitoring System

Step 6.1: Create Monitor Script (Inside VM)

bash

Inside browser-isolation VM

```
cat > ~/file-monitor.py << 'EOF'
```

```
#!/usr/bin/env python3
```

```
"""
```

```
Monitor downloads folder for suspicious files
```

```
"""
```

```
import time
```

```
import logging
```

```
from watchdog.observers import Observer
```

```
from watchdog.events import FileSystemEventHandler
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

```
    format='%(asctime)s - %(levelname)s - %(message)s',
```

```
    handlers=[
```

```
        logging.FileHandler('/home/browser/monitor.log'),
```

```
        logging.StreamHandler()
```

```
    ]
```

```
)
```

```
class DownloadMonitor(FileSystemEventHandler):
```

```
    def __init__(self):
```

```
        self.suspicious_exts = [
```

```
            '.exe', '.dll', '.bat', '.cmd', '.ps1',
```

```
            '.vbs', '.js', '.jar', '.app'
```

```
        ]
```

```
    def on_created(self, event):
```

```
        if event.is_directory:
```

```
            return
```

```
        file_path = event.src_path
```

```
        logging.info(f"📁 NEW FILE: {file_path}")
```

```
        # Check extension
```

```
        if any(file_path.endswith(ext) for ext in self.suspicious_exts):
```

```
            logging.warning(f"⚠️ SUSPICIOUS: {file_path}")
```

```
            # Add YARA scanning here
```

```
    def on_modified(self, event):
```

```
        if not event.is_directory:
```

```
            logging.info(f"✏️ MODIFIED: {event.src_path}")
```

```
    def on_deleted(self, event):
```

```
        if not event.is_directory:
```

```
            logging.info(f"🗑️ DELETED: {event.src_path}")
```

```
if __name__ == "__main__":
    path = "/home/browser/Downloads"

    event_handler = DownloadMonitor()
    observer = Observer()
    observer.schedule(event_handler, path, recursive=True)

    logging.info(f"👁 Monitoring: {path}")
    observer.start()

    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()

    observer.join()
```

EOF

```
chmod +x ~/file-monitor.py
```

```
# Create systemd service for auto-start
```

```
sudo tee /etc/systemd/system/file-monitor.service << EOF
```

```
[Unit]
```

```
Description=File Download Monitor
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=browser
```

```
ExecStart=/usr/bin/python3 /home/browser/file-monitor.py
```

```
Restart=always
```

```
[Install]
```

```
WantedBy=multi-user.target
```

EOF

```
# Enable service
```

```
sudo systemctl enable file-monitor
```

```
sudo systemctl start file-monitor
```

Part 7: Automation and Templates

Step 7.1: Create VM Template

Once VM is configured perfectly:

1. Shutdown VM
2. In Proxmox UI → Right-click VM
3. Select "Convert to template"
4. Confirm

Now you can clone this template instantly:

1. Right-click template
2. "Clone"
3. Set new VM ID and name
4. Mode: "Linked Clone" (faster) or "Full Clone"
5. Click "Clone"
6. New isolated browser VM ready in seconds!

Step 7.2: API Automation Script

bash

On Proxmox host

```
cat > /usr/local/bin/create-browser-vm.sh << 'EOF'
```

```
#!/bin/bash
```

```
# Create new browser isolation VM from template
```

```
VM_ID=$1
```

```
VM_NAME="browser-$VM_ID"
```

```
TEMPLATE_ID=100
```

```
if [ -z "$VM_ID" ]; then
```

```
    echo "Usage: $0 <vm_id>"
```

```
    exit 1
```

```
fi
```

```
# Clone from template
```

```
qm clone $TEMPLATE_ID $VM_ID \
```

```
    --name $VM_NAME \
```

```
    --full
```

```
# Start VM
```

```
qm start $VM_ID
```

```
echo "✅ Created VM $VM_ID ($VM_NAME)"
```

```
echo "    Access via SPICE: spice://192.168.1.100:$((61000 + VM_ID))"
```

```
EOF
```

```
chmod +x /usr/local/bin/create-browser-vm.sh
```

```
# Usage:
```

```
# ./create-browser-vm.sh 101
```

```
# ./create-browser-vm.sh 102
```

Part 8: Desktop Launcher (On Client Machine)

Step 8.1: Create Launcher Script

bash

On your desktop computer (not Proxmox)

```
cat > ~/launch-proxmox-browser.sh << 'EOF'
```

```
#!/bin/bash
```

```
PROXMOX_IP="192.168.1.100"
```

```
VM_ID="100"
```

```
echo "🚀 Launching isolated browser..."
```

```
# Check if VM is running
```

```
VM_STATUS=$(ssh root@$PROXMOX_IP "qm status $VM_ID")
```

```
if echo "$VM_STATUS" | grep -q "stopped"; then
```

```
    echo "Starting VM..."
```

```
    ssh root@$PROXMOX_IP "qm start $VM_ID"
```

```
    sleep 10
```

```
fi
```

```
# Open SPICE viewer
```

```
remote-viewer spice://$PROXMOX_IP:61000 &
```

```
echo "✅ Connected to isolated browser"
```

```
EOF
```

```
chmod +x ~/launch-proxmox-browser.sh
```

Step 8.2: Desktop Entry

bash

```
cat > ~/.local/share/applications/proxmox-browser.desktop << EOF
```

```
[Desktop Entry]
```

```
Version=1.0
```

```
Type=Application
```

```
Name=Isolated Browser (Proxmox)
```

```
Comment=Launch browser in Proxmox VM
```

```
Exec=$HOME/launch-proxmox-browser.sh
```

```
Icon=computer
```

```
Terminal=false
```

```
Categories=Network;WebBrowser;Security;
```

```
EOF
```

```
update-desktop-database ~/.local/share/applications/
```

Part 9: Advanced Features

Step 9.1: Snapshot Management

```
bash

# Create snapshot (via CLI)
ssh root@192.168.1.100
qm snapshot 100 clean-state

# List snapshots
qm listsnapshot 100

# Rollback to snapshot
qm rollback 100 clean-state

# Delete snapshot
qm delsnapshot 100 clean-state
```

Step 9.2: Automated Snapshot Before Each Session

```
bash

# On Proxmox host
cat > /usr/local/bin/browser-session.sh << 'EOF'
#!/bin/bash

VM_ID=100
SNAPSHOT_NAME="pre-session-$(date +%Y%m%d-%H%M%S)"

# Create snapshot
qm snapshot $VM_ID $SNAPSHOT_NAME

# Start VM
qm start $VM_ID

echo "✅ Session started with snapshot: $SNAPSHOT_NAME"
echo "    To restore: qm rollback $VM_ID $SNAPSHOT_NAME"
EOF

chmod +x /usr/local/bin/browser-session.sh
```

Part 10: Monitoring and Logging

Step 10.1: Centralized Log Collection

```
bash
```

```
# On Proxmox host
```

```
cat > /usr/local/bin/collect-vm-logs.sh << 'EOF'
```

```
#!/bin/bash
```

```
VM_ID=100
```

```
VM_IP="192.168.1.101" # Adjust to your VM's IP
```

```
LOG_DIR="/var/log/browser-isolation"
```

```
mkdir -p $LOG_DIR
```

```
# Copy logs from VM
```

```
scp browser@$VM_IP:/home/browser/monitor.log \
```

```
$LOG_DIR/vm-$VM_ID-$(date +%Y%m%d).log
```

```
echo "✅ Logs collected to $LOG_DIR"
```

```
EOF
```

```
chmod +x /usr/local/bin/collect-vm-logs.sh
```

```
# Add to cron (run daily)
```

```
echo "0 2 * * * /usr/local/bin/collect-vm-logs.sh" | crontab -
```

Configuration Summary

Proxmox Host:

```
/usr/local/bin/
├─ create-browser-vm.sh      # VM creation
├─ browser-session.sh       # Snapshot + start
└─ collect-vm-logs.sh       # Log collection
```

Inside VM:

```
/home/browser/
├─ file-monitor.py          # File monitoring
├─ monitor.log              # Activity logs
└─ Downloads/               # Monitored folder
```

Client Computer:

```
~/
├─ launch-proxmox-browser.sh
└─ .local/share/applications/
    └─ proxmox-browser.desktop
```

Quick Start Commands

```
bash

# === On Proxmox Host ===

# Create new VM from template
ssh root@192.168.1.100
/usr/local/bin/create-browser-vm.sh 101

# Start VM with snapshot
/usr/local/bin/browser-session.sh

# === On Client Computer ===

# Launch browser
~/launch-proxmox-browser.sh

# Or use desktop launcher
# (Search for "Isolated Browser" in applications)
```

Expected Performance

- **VM Boot Time:** 30-60 seconds (full Ubuntu boot)
 - **Display:** Excellent via SPICE (near-native)
 - **Snapshot Creation:** 1-2 seconds
 - **Clone from Template:** 5-10 seconds
 - **Resource Usage:** 4GB RAM, 32GB disk per VM
-

Advantages & Disadvantages

✅ **Advantages:**

- **Full hypervisor platform** with web UI
- Excellent SPICE display (best quality)
- Easy snapshot/restore/clone
- Template-based deployment
- **Enterprise-grade** management
- Can run multiple isolated VMs
- Web-based management
- Perfect for testing multiple configs

✗ **Disadvantages:**

- Requires dedicated server/machine
 - Slowest boot time (~30-60s)
 - Heaviest resource usage
 - More complex setup
 - Overkill for single-user scenarios
-

When to Use Proxmox

Best for:

- Testing multiple isolation configurations
- Managing multiple isolated browser VMs
- Enterprise deployment scenarios
- When you have dedicated hardware
- Training/demonstration purposes
- Research projects needing reproducibility

Not ideal for:

- Single laptop deployment
 - Minimal resource usage
 - Fastest boot times
 - Embedded in applications
-

Troubleshooting

Can't access Web UI:

```
bash

# Check Proxmox status
systemctl status pve-cluster
systemctl status pvedaemon
systemctl status pveproxy

# Restart services
systemctl restart pveproxy
```

VM won't start:

```
bash

# Check VM status
qm status 100

# View VM config
qm config 100

# Check logs
journalctl -u qemu-server@100
```

SPICE not connecting:

- 1. Ensure Display is set to "SPICE" in VM Hardware
- 2. Check firewall allows port 3128 and 61000+
- 3. Try noVNC console instead (via web UI)

This completes Proxmox setup! 🎉

Comparison to Other Options

Feature	Proxmox	QEMU MicroVM	Cloud Hypervisor	Firecracker
Setup Difficulty	Medium	Medium	Hard	Hard
Boot Time	30-60s	400-600ms	100-150ms	125ms
GUI Quality	★★★★★★	★★★★★★	★★★★	★★
Management	Web UI	CLI	CLI	CLI
Best Use	Multi-VM	Balance	Modern	Speed